Detecting Data Leaks via SQL Injection in AWS

Professional Technical Report

Cloud Computing Project Team

April 19, 2026

Contents

1 Abstract **3**

2 Introduction **3**

3 Background: SQL Injection and Cloud Security **3**

3.1 What SQL Injection Is . . . 3

3.2 Why SQL Injection Happens . . . 4

3.3 OWASP Context . . . 4

3.4 Real-World Relevance . . . 4

4 System Architecture and Setup **5**

4.1 Deployment Topology . . . 5

4.2 Environment Details . . . 5

4.3 Detection and Logging Components . . . 5

5 Threat Model (STRIDE Framework) **6**

6 Implementation: Attack Simulation (Section 4) **6**

6.1 Testing Methodology . . . 6

6.2 Exact Attack Commands (SQLMap) . . . 6

6.3 Exact Manual Payload URLs Used . . . 7

6.4 Leaked Data and Exposure Scope . . . 7

6.5 Result Comparison by Attack Type . . . 8

6.6 WAF Effectiveness and Residual Risk . . . 8

6.7 Required Screenshot Placeholders for Final Submission . . . 9

7 WAF and Network Defense Analysis **9**

7.1 Controls Implemented . . . 9

7.2 Before/After Comparison . . . 9

8 Detection Engine and Alerting Pipeline **10**
8.1 Lambda Detection Logic 10
8.2 Logging and Retention 10

9 Discussion **10**

10 Conclusion and Recommendations **10**
10.1 Conclusion 10
10.2 Recommendations 11

1 Abstract

This report presents the design, implementation, and evaluation of a cloud-native security pipeline for SQL injection (SQLi) attacks in Amazon Web Services (AWS). The project intentionally deploys a vulnerable Flask application backed by Amazon RDS MySQL, then measures how effectively layered cloud controls detect and mitigate data exfiltration attempts. The implementation combines AWS WAF, API Gateway, Kinesis, Lambda, CloudWatch, SNS, and S3 to create an end-to-end detection and response flow. Three SQLi attack classes were executed: union-based, boolean-based blind, and error-based attacks. Results show that direct data leakage is possible without defenses, while managed and custom WAF controls substantially reduce exploit success. The report documents payloads, observed outcomes, leaked data fields, and operational constraints such as rate-limit evaluation windows. It also maps risks through STRIDE and proposes hardening actions, including parameterized queries, improved WAF custom signatures, and continuous detection tuning.

2 Introduction

SQL injection remains one of the most impactful application security weaknesses because it targets the core trust boundary between user-controlled input and backend database queries. In this project, the team built a realistic AWS pipeline to show how SQLi attacks can be launched, detected, blocked, logged, and surfaced to operators. The architecture is intentionally layered so that each control addresses different portions of the kill chain:

- Network and application edge filtering (API Gateway + WAF).
- Behavioral detection and classification (Lambda regex engine).
- Alerting and observability (SNS + CloudWatch dashboard).
- Forensic persistence (Kinesis + S3 archival).

The objective is not only to demonstrate exploitation, but to measure defense effectiveness and identify residual risk. The report therefore emphasizes evidence-driven comparisons between pre-WAF and post-WAF behavior and highlights where managed-rule signatures can still be evaded.

3 Background: SQL Injection and Cloud Security

3.1 What SQL Injection Is

SQL injection is a vulnerability class in which attacker-controlled input is interpreted as part of a database query rather than as inert data. When an application constructs

SQL statements by concatenating raw input, attackers can alter query logic, bypass authentication, enumerate schema metadata, or extract sensitive records. SQLi can impact confidentiality, integrity, and availability in a single technique family.

In the tested application, vulnerable endpoints accepted query parameters and interpolated them into SQL statements. This allowed both authentication bypass attempts and data extraction attempts against user records.

3.2 Why SQL Injection Happens

From a secure coding perspective, SQLi is a failure of input handling and query construction discipline. Typical root causes include:

- Unsanitized user input reaching query builders.
- Direct string concatenation for SQL statements.
- Lack of prepared statements and parameter binding.
- Over-privileged database accounts that expand blast radius.
- Missing input validation and weak output/error handling.

Even if an application hides SQL errors from users, attackers can still use inferential techniques such as boolean-blind injection and time-based probing to reconstruct data. Therefore, secure design must address both obvious and subtle leakage paths.

3.3 OWASP Context

SQL injection remains a foundational issue within modern application security risk models and is strongly aligned with the OWASP Top 10 category focused on injection flaws. OWASP rankings evolve over editions, but injection-related weakness categories continue to represent high-likelihood and high-impact risks because of their direct path to sensitive data compromise. In cloud environments, SQLi can be amplified by misconfigured identity permissions, centralized data stores, and internet-exposed APIs.

3.4 Real-World Relevance

Historically, major incidents involving large-scale data exposure have included injection-style weaknesses or adjacent input-validation failures. Common lessons from widely discussed breaches include:

- Public-facing endpoints frequently become initial access points.
- Attackers pivot from one vulnerable query to broad data access.
- Weak logging and delayed detection increase dwell time.

- Compliance and reputational damage often exceed immediate technical loss.

The practical takeaway is that SQLi defense must be multi-layered: secure code, preventive edge controls, runtime detection, and strong operational response.

4 System Architecture and Setup

4.1 Deployment Topology

The project architecture follows a five-layer model: attack simulation, network defense, detection engine, logging pipeline, and alerting/dashboard. The core traffic path is:

$$\text{Attacker} \rightarrow \text{API Gateway} \rightarrow \text{WAF} \rightarrow \text{Flask App (EC2)} \rightarrow \text{RDS MySQL}$$

In parallel, telemetry is streamed to Kinesis and Lambda for analysis, then persisted in DynamoDB/S3 and visualized via CloudWatch and a frontend dashboard.

4.2 Environment Details

- Target host: 3.144.240.191 (Flask app on port 5000).
- Attacker host: 18.221.229.40.
- Database: Amazon RDS MySQL, database name `userdata`.
- Vulnerable endpoints: `/login` and `/search`.
- API endpoint used in defense tests: https://2y68q1dt27.execute-api.us-east-2.amazonaws.com/prod.

4.3 Detection and Logging Components

The Lambda detector uses regex-based pattern classification with severities (LOW, MEDIUM, HIGH, CRITICAL) and writes findings to DynamoDB. High-severity events trigger SNS notifications. Firehose/Kinesis settings use buffered delivery to S3, enabling historical investigation and long-term retention policies.

5 Threat Model (STRIDE Framework)

Table 1: STRIDE Threat Mapping for This Project

STRIDE Category	**Threat Focus**	**Example in This Project**
Spoofing	Identity impersonation	Attacker attempts login bypass using SQLi patterns to authenticate as privileged user.
Tampering	Data integrity compromise	Injection payloads could execute unauthorized updates/deletes if write queries are reachable.
Repudiation	Non-repudiation failure	Without reliable audit logs, attackers can deny malicious requests.
Information Disclosure	Confidentiality breach	UNION-based extraction leaks sensitive columns such as full name, email, SSN, and credit card.
Denial of Service	Availability degradation	Time-delay or heavy-query payloads (for example sleep-based probes) increase latency and can exhaust resources.
Elevation of Privilege	Access control bypass	Metadata enumeration (for example information_schema) helps attackers discover additional tables and privileges.

6 Implementation: Attack Simulation (Section 4)

6.1 Testing Methodology

Attack simulation combined automated SQLMap campaigns and manual payload tests. SQLMap was selected to ensure repeatability across techniques while manual URLs were used to validate edge behaviors and direct endpoint responses.

The three attack classes executed were:

1. Union-based SQL injection.
2. Boolean-based blind SQL injection.
3. Error-based SQL injection.

6.2 Exact Attack Commands (SQLMap)

Listing 1: Union-based SQLi command

```
python3 sqlmap.py -u "http://3.144.240.191:5000/search?name=test" --technique=
    U --dbs --dump --batch -v 3
```

Listing 2: Boolean-based blind SQLi command

```
python3 sqlmap.py -u "http://3.144.240.191:5000/login?username=admin&password=
    test" --technique=B -D userdata -T users --dump --batch -v 2
```

Listing 3: Error-based SQLi command

```
python3 sqlmap.py -u "http://3.144.240.191:5000/login?username=admin&password=
    test" --technique=E --dump-all --batch
```

6.3 Exact Manual Payload URLs Used

Table 2: Manual Attack URLs Recorded in the Lab

Attack Case	Payload / URL
Login bypass test	http://3.144.240.191:5000/login?username=admin%27--&password=x
UNION extraction test	http://3.144.240.191:5000/search?name=%27%20UNION%20SELECT%20id,full_name,ssn%20FROM%20users--
Table enumeration test	http://3.144.240.191:5000/search?name=%27%20UNION%20SELECT%201,table_name,3%20FROM%20information_schema.tables--
Time delay test	http://3.144.240.191:5000/login?username=admin%27%20AND%20SLEEP(5)--&password=x

6.4 Leaked Data and Exposure Scope

Across successful pre-defense runs, the following high-risk fields were reported as exposed:

- full_name
- email
- ssn
- credit_card
- dob

From a compliance perspective, SSN and payment-related fields indicate direct personally identifiable information (PII) and potential financial-data exposure.

6.5 Result Comparison by Attack Type

Table 3: Attack Results, Timing, and WAF Outcome Matrix

Attack Type	Payload Pattern	Data Leak (No WAF)	Approx. Time Taken	Blocked by WAF?
Union-based	UNION SELECT	Yes: user records including PII fields	Placeholder: capture from SQLMap summary screen	Yes, by AWSManagedRulesSQLiRuleSet
Boolean blind	true/false inference	Yes: inferred user data	Placeholder: capture from SQLMap summary screen	Yes, via custom rate limit behavior
Error-based	DB error exploitation	Yes: table and schema details	Placeholder: capture from SQLMap summary screen	Partially mitigated by managed rules; verify with screenshots
Time-delay probe	SLEEP(5)	No direct row dump, but service latency impact observed	5 seconds in manual test endpoint response	Yes, by AWSManagedRulesKnownBadInputs

6.6 WAF Effectiveness and Residual Risk

Defense testing showed strong improvement after WAF controls were enabled:

- UNION SELECT payloads moved from data leakage to explicit blocking.
- Always-true payloads with OR 1=1 were blocked by SQLi rule sets.
- Time-delay signatures were blocked by known-bad-inputs coverage.

However, testing also exposed two operational caveats:

1. A minimal payload such as `admin'-` could evade managed signatures when no common SQL keyword token was present.
2. Custom rate limits were not instantaneous and appeared to apply over AWS evaluation windows (roughly 30 to 60 seconds), creating a short burst window before enforcement.

These findings emphasize that managed WAF rules are necessary but not sufficient. They should be augmented with custom signatures and application-layer hardening.

6.7 Required Screenshot Placeholders for Final Submission

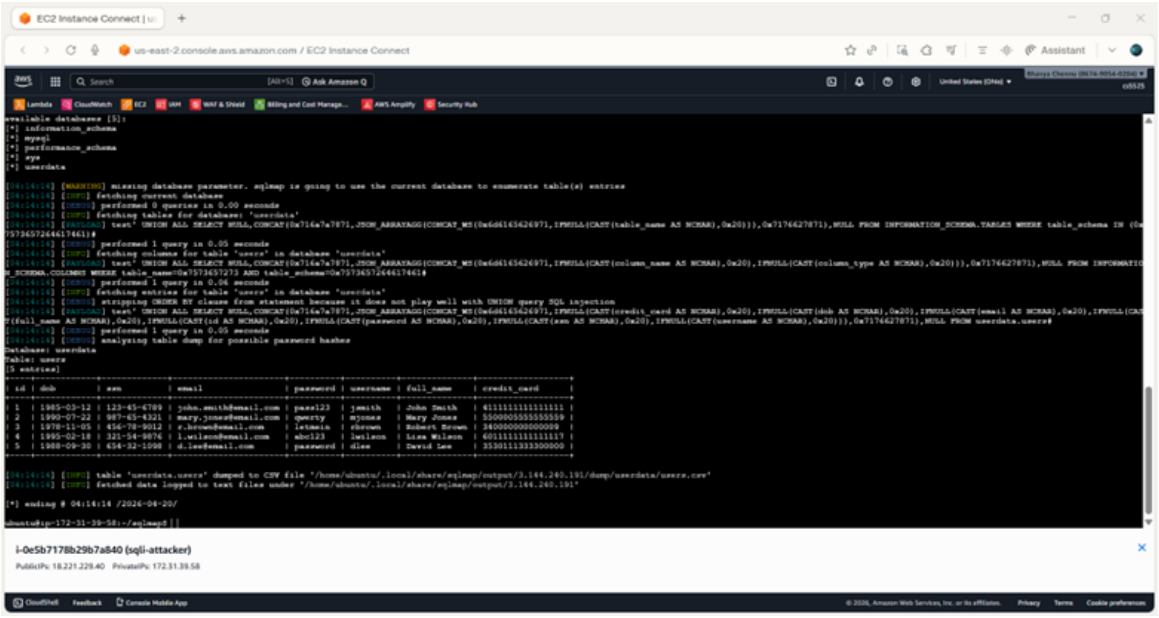

Figure 1: Figure A. SQLMap extraction output from the attack simulation environment

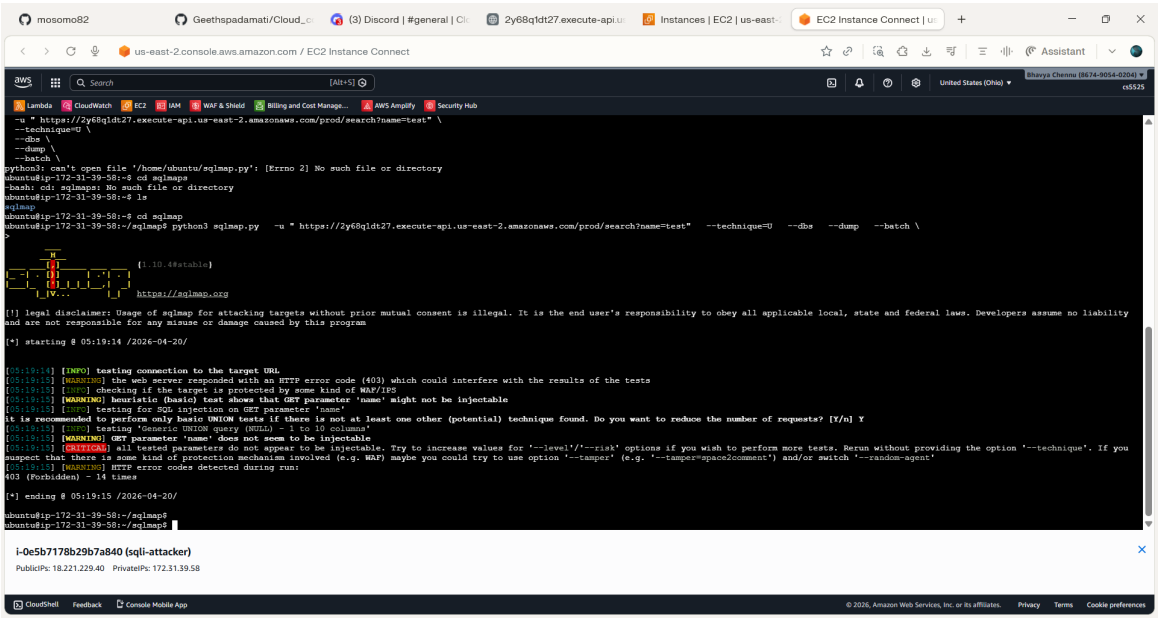

Figure 2: Figure B. WAF 403 Forbidden block response during SQLi payload test

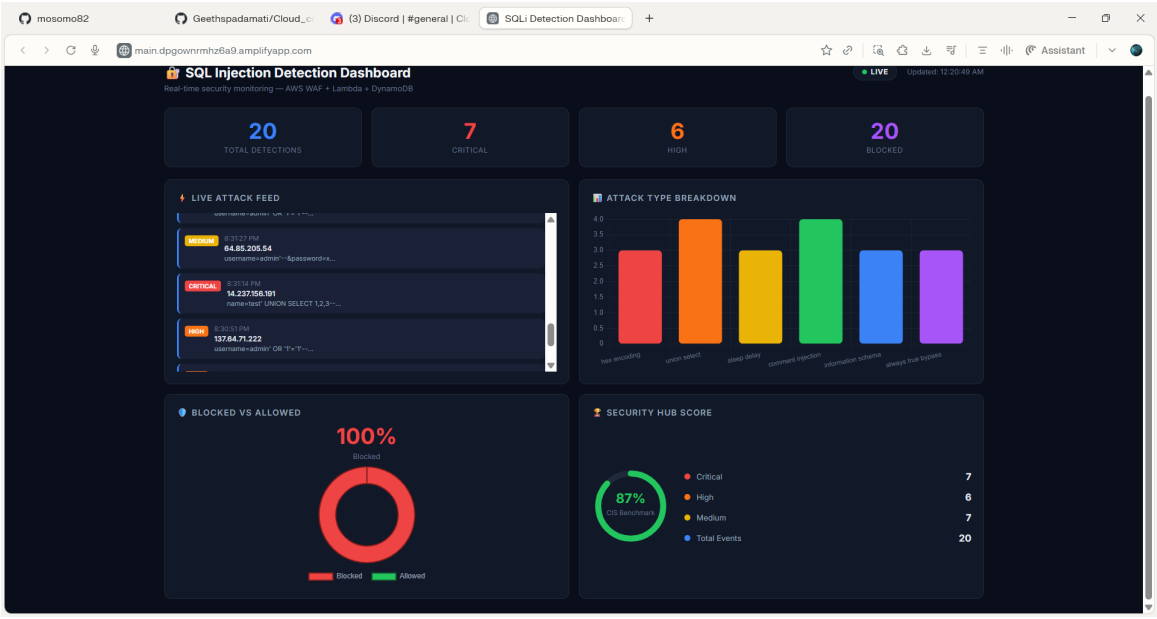

Figure 3: Figure C. CloudWatch security dashboard during active attack simulation

7 WAF and Network Defense Analysis

7.1 Controls Implemented

Network defense centered on API Gateway fronting and AWS WAF managed rule groups with custom rate limiting. Key observed controls included SQLi-focused managed rules and bad-input signatures.

7.2 Before/After Comparison

Table 4: Observed WAF Comparison Results

Attack Type	Without WAF	With WAF	Rule/Behavior
UNION SELECT	Leaked data	Blocked	AWSManagedRulesSQLiRuleSet
Always true OR 1=1	Login bypassed	Blocked	AWSManagedRulesSQLiRuleSet
Boolean blind burst	Leaked data	Blocked after thresholding	Custom rate-limit rule
SLEEP() time delay	5-second delay observed	Blocked	AWSManagedRulesKnownBadInpu
Normal request	Allowed	Allowed	Expected allow behavior

8 Detection Engine and Alerting Pipeline

8.1 Lambda Detection Logic

The Lambda function classifies incoming request traces with regex rules mapped to severities. Examples include `union_select` and `information_schema` at CRITICAL severity, and `sleep_delay` at HIGH severity. Findings are persisted to DynamoDB and high-severity findings are published to SNS.

This produces two operational advantages:

- Fast triage through severity labels.
- Longitudinal visibility through structured finding records.

8.2 Logging and Retention

Firehose/Kinesis settings provide buffered log delivery to S3 partitions by date. Lifecycle policy then transitions storage classes over time and expires old objects, balancing forensic value and storage cost.

9 Discussion

The project demonstrates that SQLi remains practical against insecure query construction, especially in lab environments intentionally designed for educational exploitation. The key result is that defense-in-depth meaningfully changes outcomes: successful pre-defense exfiltration becomes largely blocked post-defense, while detection and alerting reduce mean time to awareness.

At the same time, edge-case bypass behavior highlights an important lesson. Security controls built on pattern signatures can miss low-context payloads, and timing-based controls may introduce delayed enforcement windows. Consequently, secure coding remains the primary control and WAF should be treated as a protective layer rather than a substitute for parameterized queries.

10 Conclusion and Recommendations

10.1 Conclusion

This implementation achieved an end-to-end cloud security workflow for SQL injection: simulate attacks, observe impact, apply controls, monitor outcomes, and document residual risk. The architecture successfully integrated preventive, detective, and response mechanisms across AWS services.

10.2 Recommendations

1. Refactor application queries to use prepared statements and parameter binding everywhere.
2. Add targeted custom WAF signatures to catch short evasive payloads such as comment-only bypass attempts.
3. Expand automated regression tests to include both exploit payloads and false-positive controls.
4. Track detection quality metrics explicitly: precision, recall, and alert latency.
5. Preserve immutable evidence artifacts (screenshots, command output, response logs) for every test case in the final report package.

Appendix A: Submission Checklist

- Exact payload/URL table included.
- Screenshot placeholders included and labeled for replacement.
- Leaked data fields listed: full_name, email, ssn, credit_card.
- Attack duration column included (replace placeholders with SQLMap terminal timings).
- WAF blocked/not-blocked outcomes documented per attack type.

Appendix B: Reference Artifacts Used

- attack-simulation/docs/attack-playbook.md
- attack-simulation/results/attack-summary.md
- defense/results/waf-comparison-results.md
- detection/lambda/handler.py
- infrastructure/cloudwatch.tf
- infrastructure/kinesis.tf
- infrastructure/s3.tf
- README.md